Beeman Numerical Integration Multi-Body Gravitational Simulation

Ben Morrison

March 29, 2024

1 Introduction

The aim of the project is to design a python program in an objected oriented programming style that can simulate the motion of a multi-body system under the influence of gravity. The program has to take in information from a .txt file that contains initial conditions for an arbitrary number of bodies. The program uses both the Beeman and Euler direct step numerical integration methods to compute the positions and velocities of the bodies in the program and the gravitational force law is used, in both cases, to calculate the accelerations. The program must be able to perform three experiments - which are explained in detail in section 2.2. The project contains 5 files; run.py, solar.py, planet.py, planet_euler.py and parameters-solar.txt. To run the program all files must be in the same directory - then run.py can be run. Further instructions will be printed to the console.

2 Methods

The program has been constructed in an object-oriented programming style and thus has it's functionality embedded into classes.

2.1 Classes

The Planet Class

The planet class contains variables for the specific bodies name, mass, orbital radius, colour to be plotted and the measured orbital period. Information for the measured orbital period come from NASA, [1] and [2]. The planet class contains methods which apply the Beeman numerical integration method to update the bodies position and velocity each small timestep, measured in Earth years.

$$\underline{r}(t+\Delta t) = \underline{r}(t) + \underline{v}(t)\Delta t + \frac{1}{6}[4\underline{a}(t) - \underline{a}(t-\Delta t)]\Delta t^2 \tag{1}$$

$$\underline{v}(t+\Delta t) = \underline{v}(t) + \frac{1}{6}[2\underline{a}(t+\Delta t) + 5\underline{a}(t) - \underline{a}(t-\Delta t)]\Delta t \tag{2}$$

and the acceleration updated via the Gravitational Force law,

$$\begin{aligned} \underline{F_1} &= G\frac{m_1 m_2}{|\underline{r}_{21}(t)|^2}\hat{\underline{r}} \\ \underline{a} &= \frac{F_1}{m_1} \\ &= G\frac{m_2}{|\underline{r}_{21}(t)|^2}\hat{\underline{r}} \end{aligned} \tag{3}$$

where $\underline{r}_{12} = r_{12}\hat{\underline{r}}$ is the displacement vector between two bodies, labelled 1 and 2. The Planet class also includes a method to calculate the current kinetic energy of the body, via

$$E_k = \frac{1}{2}m(\underline{v}\cdot\underline{v}) \tag{4}$$

and a method to check if the planet has performed one full revolution (one year) by checking when the planet crosses the negative x-axis (when it's y-coordinate in the previous timestep was negative and in the current timestep is positive - this isolates full revolutions since at a half revolution the previous y-coordinate will be postiive and the current will be negative).

The Planet Euler Class
The Planet Euler class is inherited from the Planet class and has updated methods for calculating the position and velocity of the body, via the Euler direct step method,

$$\underline{r}(t + \Delta t) = \underline{r}(t) + \underline{v}(t)\Delta t \tag{5}$$
$$\underline{v}(t + \Delta t) = \underline{v}(t) + \underline{a}(t)\Delta t \tag{6}$$

This class is used exclusively for the second experiment.

The Solar Class
Each experiment has it's own method in the Solar class which is called in the run.py file after user input is parsed. For performance reasons, a boolean value for each experiment is introduced so that only the information required for that specific experiment is appended to lists. Without this check the simulation timestep has to be lowered to avoid unexpected behaviour, such as planets flying off to infinity or the simulation producing inaccurate and inconsistent orbital periods. The program will only run one experiment at a time, after said experiment has concluded information will be printed to the console and, if necessary, a graph will be plotted via the matplotlib.pyplot library. The program must be restarted in order to perform another experiment. It should also be noted that the solar class has a method which computes the total energy of the system,

$$\begin{aligned} E_T &= \sum E_K + \sum E_P \\ &= \sum_{i=0}^{N} \frac{1}{2} m(\underline{v_i} \cdot \underline{v_i}) + \sum_{k \neq j} \left(-G \frac{m_j m_k}{r_{jk}} \right) \end{aligned} \tag{7}$$

and a class which computes the angle, θ of the planet relative to the unit vector that lies on the x-axis, $\underline{e_1} = (1, 0)$ using the dot product,

$$\begin{aligned} \underline{r_i} \cdot \underline{e_1} &= |\underline{r_i}||\underline{e_1}| cos(\theta) \\ \theta &= cos^{-1} \left(\frac{r_i \cdot \underline{e_1}}{r_i} \right) \end{aligned} \tag{8}$$

where the position of the ith body relative to the sun, $\underline{r_i}$, is defined as,

$$\underline{r_i} = \underline{r^{'}} - \underline{r_{sun}} \tag{9}$$

with $\underline{r^{'}}$ being the position relative to the origin at (0, 0).

2.2 Experiments

Standard Setup
All of the experiments are conducted with the innermost planets, up to and including Jupiter, orbiting around the sun. Information for these bodies was obtained from NASA, [1].

Experiment 1: Orbital Periods
The aim of experiment one is to simulate the orbital periods of the inner planets, up to Jupiter, and compare this to their real life measured values. Experiment one runs for a total simulation time of 160 years - chosen because the simulation needs time to reach steady state. The total number of orbital periods for each body is printed to the console every time a year passes for that body. The total energy of the system is also printed to the console every Earth year. At the end of the simulation the last value for the orbital period is taken and compared to the actual value. The difference, both absolute and as a percentage, are printed to the console for each body after the simulation has completed 160 Earth years. The experiment is run with a timestep of 0.005 Earth years for 32000 iterations.

Experiment 2: Energy Conservation and Alternative Integration Methods
The aim of experiment two is to deduce which of the Beeman or Euler direct step numerical integration methods conserves energy more accurately. The experiment consists of two simulations, each corresponding to one of the integration methods, which run sequentially. The total energy of the system is recorded every Earth year, for a total of 400 Earth years. After 400 Earth years, an energy vs time graph containing the data from both experiments is plotted. The experiment is run with a timestep of 0.02 Earth years for 20000 iterations.

Experiment 3: Planetary Alignment
The aim of experiment three is to determine the average time separation between planetary alignments for the innermost planets, up to Jupiter. Planet's are only considered aligned if their centre axis are all at the same angle, measured anti-clockwise from the x-axis. Below the alignment condition has been graphically represented. The planetary alignment angle has been chosen to have a threshold of ±5%,

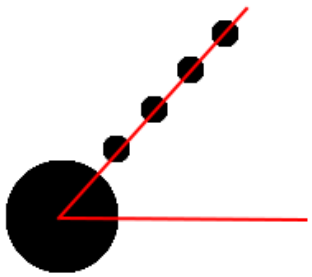


Figure 1: Planetary alignment condition

any lower threshold value and the simulation has to run for a very long time to see any alignments. The simulation will run for 3000 Earth years. The total elapsed time is printed to the console every 100 Earth years and, after the first alignment, the total alignments and time since last alignment are also printed. When an alignment is detected a message will be printed to the console to indicate that this has occurred. After 3000 Earth years a graph of time between alignments vs total elapsed time will be plotted and the mean time between alignments will be printed to the console. The experiment is run with a timestep of 0.02 Earth years for 150000 iterations.

3 Results

3.1 Experiment 1

Results from experiment 1 are displayed below in tabular form with units of Earth years. The first table is for a simulation of 80 Earth years and the second is a simulation of 160 Earth years, both at the same timestep.

	Mercury	Venus	Earth	Mars	Jupiter
Simulated Period	0.245	0.615	1.000	1.880	11.820
Actual Period	0.241	0.616	1.001	1.882	11.866
Difference	0.004	0.001	0.005	0.002	0.046
Difference (%)	1.56	0.10	0.05	0.12	0.39

	Mercury	Venus	Earth	Mars	Jupiter
Simulated Period	0.240	0.615	0.995	1.880	11.820
Actual Period	0.241	0.616	1.001	1.882	11.866
Difference	0.001	0.001	0.006	0.002	0.050
Difference (%)	0.46	0.10	0.60	0.12	0.39

By observing the percentage difference for Mercury it can be seen that the accuracy has improved as simulation time increased. The opposite is true for Earth's period, which became much more inaccurate. This is due to simulation having an optimal running time. If the experiment is not run for long enough (in simulation time) then the planets will not reach a steady state and thus the periods will be inaccurate. If the simulation is run for too long then the difference in orbital period will compound and become very high. Which is shown in the below table of results for an 800 Earth year simulation.

	Mercury	Venus	Earth	Mars	Jupiter
Simulated Period	0.240	0.610	0.995	1.870	11.820
Actual Period	0.241	0.616	1.001	1.882	11.866
Difference	0.001	0.006	0.006	0.012	0.046
Difference (%)	0.46	0.92	0.56	0.65	0.39

3.2 Experiment 2

Results from experiment 2 are displayed in figure 2, which depicts a graph of total energy vs time for the Beeman and Euler direct step numerical integration methods. Both simulations were run with a timestep of 0.02 for a total time of 400 Earth years. The data shows that the energy oscillates around a range

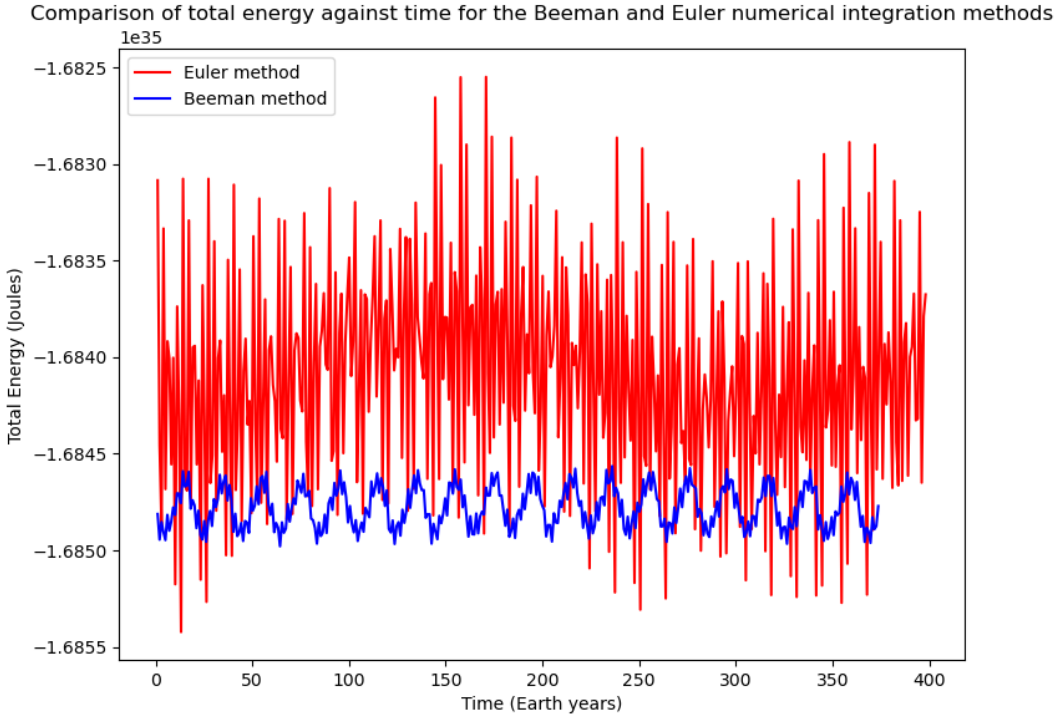


Figure 2: Graphical representation of experiment 2 results

of $\approx 0.0025 \cdot 10^{35}$ Joules for the Euler method and $\approx 0.0001 \cdot 10^{35}$ Joules for the Beeman method. The difference in these ranges are significant and indicate that the Beeman method conserves energy better than the Euler method.

3.3 Experiment 3

Results from experiment 3 are displayed in figure 3, which depicts a graph of time between alignments against the total time elapsed. The calculated mean value of time between alignments is 313.4 Earth years. This value is not very useful as the graph shows: the time between alignments is never actually close to 300 years, instead it shows large differences varying from ≈ 100 years to ≈ 700 years. It can be seen that the data does not follow any clear trend for a simulation period of 3000 Earth years.

4 Discussion

4.1 Experiment 1 Discussion

The results from experiment 1 indicate that the simulation is accurate when run for the "optimal" time, which is somewhere between 160 years and 800 years. If the simulation were run for a time outwith this range, the results differ by more than 1% and are thus unsuitable for real world use. Since the experiment

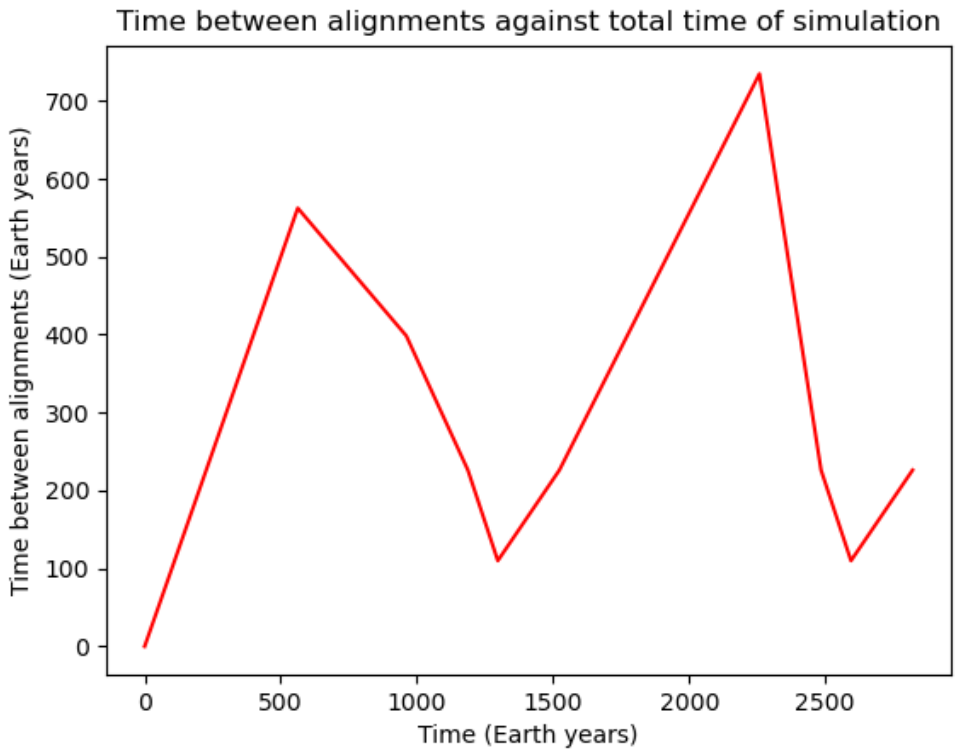


Figure 3: Graphical representation of experiment 3 results

initializes the planets in their average radial orbits, the semi-major axis of the orbit will be smaller than the real observed value and thus the difference from the real orbital periods will gradually become larger. This can be avoided if the planets are initialized on their real semi-major axis and thus replicating their real life orbits closer.

4.2 Experiment 2 Discussion

The qualitative result of experiment 2 is exactly the expected result, that the Beeman method conserves energy better than the Euler direct step method. This is because the Beeman method, by design, accounts for the conservation of energy. This experiment could be improved quantitatively if it was repeated with a lower timestep and higher number of iterations so that the trajectory, and therefore velocity, of the planets are more accurately calculated and therefore a more accurate energy can be obtained. However, this is not strictly necessary as the desired result is qualitatively shown with the aforementioned timestep and number of iterations.

4.3 Experiment 3 Discussion

If this experiment were to provide a more accurate result the simulation would have to run for a much longer number of iterations. However, as mentioned in the Experiment 1 discussion, the simulation becomes more and more inaccurate with time above ≈ 800 years. Therefore, the simulation is unsuitable for providing an accurate time between separations of the innermost planets up to Jupiter.

4.4 Conclusion

To conclude, an object-oriented program was written in python to compute the gravitational interaction between an arbitrary multi-body system. Information for the Sun and the innermost planets up to Jupiter were read by the program from a .txt file and three experiments were conducted. The results for experiment 1 and 2 were accurate and gave the expected result. The program was unsuitable for accurately predicting the average separation between planetary alignments, as required in experiment 3, and so demonstrated the limitations of a simple Beeman numerical integration based simulation for simulating the motion of the planets in the solar system.

*The project has a word count of 1653, not including this sentence.

References

[1] NASA. https://nssdc.gsfc.nasa.gov/planetary/factsheet/. *NASA Planetary Fact Sheet*, 19/03/2024 at 15:15.

[2] NASA. http://ssd.jpl.nasa.gov/horizons.cgi. *NASA Horizons System*, 27/03/2024 at 11:39.